Do Generate–Verify–Repair Guards Improve LLM NetOps Outputs? A Paired Emulator Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory experiment (n=200 natural-language NetOps intents; study_id cnd-001-5f3a) that compares ungated LLM generation to a bounded generate–verify–repair (GVR) pipeline applied to a single shared LLM candidate per intent. Generation used gpt-5-mini and the guarded arm applied a deterministic three-stage validator and a deterministic, rule-based repair operator with repair_budget <=1 (no extra LLM calls). Artifact-backed primary outcomes: baseline successes = 199/200 (0.995), guarded = 200/200 (1.000); paired contingency n11=199, n10=1, n01=0, n00=0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields p = 1.0; paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI = [0.0, 0.015]. The preregistered 0.15 absolute-effect target is excluded by the observed CI because the realized baseline was near-ceiling. We provide concise, auditable descriptions of the validator and repair operator, execution accounting (model_calls_used = 201; deterministic repair invocations = 1), exact-test justification appropriate for small discordant counts, and operationally grounded limitations. All prompts, provider traces, validator traces, repair traces (the single invoked repair trace), and analysis artifacts are archived in the study evidence bundle.**

I. Introduction

Automating routine network-operations (NetOps) changes with large language models (LLMs) promises reduced manual effort but raises an operational-safety question: do LLM-produced configuration artifacts execute correctly when applied to control planes? Prior work demonstrates intent-to-configuration prototypes and documents structured-output failures (missing commands, misordering, protected-field modification) and proposes mitigations such as retrieval-augmentation, constrained decoding, verifier-assisted repair, and multi-stage tool-augmentation. Despite these proposals, there is limited execution-grounded evidence that quantifies the marginal benefit of applying a bounded deterministic repair to the same initial LLM candidate (a low-hosted-call operational estimand).

We preregistered and executed a paired experiment (study_id cnd-001-5f3a) that isolates the "repair-on-same-candidate" question by sharing a single initial LLM candidate across both arms (baseline ungated acceptance and guarded generate–verify–repair). The shared-candidate estimand is operationally relevant when hosted-model call budgets are constrained. Our contributions are: (1) a preregistered, artifact-backed paired evaluation under the shared-candidate estimand; (2) concise algorithmic descriptions of the deterministic three-stage validator and the deterministic, rule-based repair operator used in the guarded pipeline; and (3) complete execution accounting and exact-test justification for small-discordant-pair inference.

II. Related Work

Research exploring LLMs for NetOps has produced three interlocking strands that motivate the present study. First, intent-to-configuration work demonstrates that LLMs can synthesize device CLI-like commands from natural-language intents but that structured-output errors are common without additional constraints (e.g., missing required commands, incorrect ordering, or violation of protected-field policies). Empirical intent-translation prototypes and surveys document these failure modes and report prototype success under curated conditions (Nguyen et al., 2024) [doi:10.1002/nem.2313]. Complementary surveys and architectural analyses (Bilal et al., 2026) characterize the operator-facing requirements—auditability, bounded tool use, and verifier integration—needed for operational adoption.

Second, mitigation approaches have emerged from both applied and adjacent domains. Retrieval-augmented generation (RAG), live schema grounding, and constrained or grammar-aware decoding have reduced hallucination in tasks such as NL2SQL and document-grounded generation; grammar/constrained decoding methods (e.g., Geng et al., 2023) demonstrate that enforcing syntactic or schema constraints at decode-time reduces format errors without model fine-tuning. Clinical and document RAG studies further show that grounding plus deterministic validators reduces high-stakes hallucinations (Wołk, 2025) [analysis in evidence bundle], but these techniques require careful adaptation for device-specific CLI dialects and workflow policies.

Third, deterministic formal verification and emulator-based validators are natural assurance layers. Formal-methods variants (NetKAT/weighted NetKAT and ensemble verifiers) provide provable checks for reachability and isolation properties (Suárez Acevedo et al., 2026) [doi:10.1145/3808318], and emulator-backed validation captures many syntactic and stateful correctness conditions. Prior work highlights, however, that verification has limits: control-plane dynamics (e.g., BGP convergence timing), vendor idiosyncrasies, and runtime transient rules are not always captured by static or deterministic checks.

Generate–verify–repair (GVR) architectures—where generation is followed by deterministic or model-assisted checking and then repair—are proposed as pragmatic operational patterns, but fully instrumented, execution-grounded evaluations remain scarce. Surveys and position papers call for workflow-centered benchmarks that include per-intent execution outcomes and repair traces; the literature documents proposals and partial evaluations but lacks preregistered paired measurements that isolate repair-on-the-same-candidate under a constrained model-call budget (the gap G2 noted in our evidence synthesis). Finally, adversarial-prompt research (Das et al., 2026) underscores a persistent operational risk: unconstrained LLM outputs remain attackable, increasing the imperative for auditable deterministic validators in NetOps pipelines. Our study situates itself explicitly within this empirical gap by measuring the marginal, auditable yield of a deterministic repair operator applied only to the shared initial candidate and by using an emulator+deterministic validator as the execution-grounded scoring mechanism.

III. Methodology

Preregistration and overall design: We preregistered a paired, shared-initial-generation confirmatory protocol (study_id cnd-001-5f3a). The confirmatory set contains 200 curated natural-language intents covering single-device and small multi-device changes (VLAN/MTU edits, uplink switchover, safe shutdown-configure-restore patterns). For each intent the hosted model produced a single initial candidate (shared across arms). The baseline arm accepts that candidate verbatim; the guarded arm applies a deterministic three-stage validator and, if the validator reports a repairable violation family, invokes at most one deterministic repair transformation (repair_budget <=1) implemented in code (no additional LLM calls). The primary estimand is the paired_success_rate_difference_guarded_minus_baseline and the preregistered primary hypothesis test is McNemar's paired binary test implemented in its exact-binomial form (see Statistical methods below). Planned maximum hosted-model calls = 400; realized model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). The execution environment used Dockerized Mininet + FRR images and deterministic_netops_validator_v5 as the validator; all run artifacts are archived in the evidence bundle.

Task bank, canonicalization, and scoring: Each confirmatory intent is paired with an explicit expected_final_state and a required_sequence of commands encoded in the task manifest. Scoring follows the preregistered full_intent_constraint_validation rule and requires: (a) syntactic parse and canonical normalization of CLI-like candidate lines into tokenized command records; (b) presence of every command in the task's required_sequence (MISSING_REQUIRED_COMMAND flagged otherwise); (c) ordering/dependency checks (e.g., make-before-break, shutdown_configure_restore) producing dependency violation codes when ordering constraints are unmet; (d) group-activity and minimum-active constraints (e.g., final-group constraints for uplink pairs); and (e) preservation/protection checks for interfaces marked as protected. After canonicalization the deterministic emulator applies the normalized_configuration and computes observed final_state; per-interface, per-field equality with expected_final_state is required for a passing score. The validator emits a machine-readable trace per episode containing fields such as parsed_command_count, observed_touched_field_count, normalized_configuration, violation_codes (families and deterministic subcodes), validation_after.valid, and validator_version. These traces are the inputs for repair decisions and are archived for audit.

Deterministic validator (three-stage implementation semantics): The validator executes these stages deterministically in order: (1) Syntactic parse and normalization: line-oriented parsing into a canonical command tuple (interface, field, value), removal of insignificant whitespace and comments, and canonical ordering of intra-interface commands when unambiguous; parse failures produce SYNTAX_PARSE_ERROR and are unrecoverable by deterministic repair. (2) Structural and workflow checks: this stage compares parsed commands to the task payload (required_sequence and workflow_policies), verifies ordering constraints and group-activity invariants, and emits deterministic violation families (examples observed in traces: MISSING_REQUIRED_COMMAND, DEPENDENCY_VIOLATION, PROTECTED_INTERFACE_MODIFICATION). (3) Deterministic emulator application: the normalized_configuration is applied to a deterministic control-plane emulator that updates per-interface fields; the resulting observed final_state is compared to expected_final_state using exact field equality checks for the set of fields declared in the task (admin, mtu, vlan). The validator records counts of parsed and required commands and a final boolean valid flag. All emitted traces are used both for scoring and for deterministic repair logic.

Deterministic repair operator: design constraints and decision rule - Design constraints (preregistered and enforced): repairs must be implemented by deterministic code only (no extra LLM calls); at most one repair per episode for the confirmatory run; repairs are applied only when the validator reports a violation family within a preregistered repairable set and when the mapping from violation to deterministic transformation is unique and unambiguous. - Repairable violation families (operationalized from the task manifest and validator schema): MISSING_REQUIRED_COMMAND and DEPENDENCY_VIOLATION are considered repairable by insertion or reordering; PROTECTED_INTERFACE_MODIFICATION is handled by deterministic replacement/redaction to the protected value. Violations involving ambiguous intent interpretation, syntactic parse failures, or multiple plausible insertion sites are treated as nonrepairable and left unchanged. - Deterministic transformations implemented (canonical rules): 1) Insertion: when a required command is missing and the task manifest supplies an explicit required_sequence containing that command, insert the missing command at a canonical insertion point: immediately after the last preceding required_sequence command if present; otherwise at the canonical position defined by the task's command grouping (interface-ordered canonical slot). The inserted command text is constructed deterministically from the task manifest required_sequence and normalized to the validator's canonical form. 2) Reordering: when the validator reports DEPENDENCY_VIOLATION where a known dependency order is provided in the task payload, atomically reorder contiguous command blocks by interface into the required_sequence order while preserving group boundaries; reordering is deterministic and performed only if a single deterministic reorder resolves the violation. 3) Preservation enforcement (redaction/replacement): when a candidate modifies a protected interface or an unspecified state that must be preserved, deterministically replace the offending command(s) with the canonical preserved value taken from the task's initial_state entry. - Repair decision predicate: apply a repair iff (violation_family $\in$ repairable_set) AND (deterministic_mapping_is_unique == true) AND (repair_budget_remaining > 0). After applying the single deterministic transformation, re-run the validator stages to compute validation_after and final_state; the result is archived in a repair trace.

Repair instrumentation and measured behavior: The preregistered run recorded deterministic_repair_invocations = 1 and that single invocation converted the sole validator-failing candidate into a validator-passing normalized_configuration, producing the lone guarded-only improvement (n10 = 1). Across the confirmatory set syntactic/parse failures = 0. Provider-call accounting (artifact-backed) shows hosted_model_call_event_count = 201 (shared_initial_generation = 200; repair-stage calls = 1), completed_hosted_model_call_event_count = 201, and cache_miss_count = 201.

Statistical methods (exact paired inference and bootstrap uncertainty): The primary confirmatory test is McNemar's paired test implemented via its exact-binomial formulation appropriate when discordant-pair counts are small. Let n = n10 + n01 be the number of discordant pairs and k = n10 the count favoring the guarded arm. Under H0 (no difference) each discordant pair has probability 0.5 of falling in either order, so the two-sided p-value is computed from the binomial distribution Binomial(n, 0.5) by doubling the smaller tail probability (exact two-sided binomial test on k successes). For our observed discordant counts (n10 = 1, n01 = 0 => n = 1, k = 1) the exact two-sided p-value equals 1.0 (artifact-backed p reported in analysis results). Complementary uncertainty quantification uses a paired-bootstrap percentile 95% CI on the paired difference (guarded - baseline) with 10,000 resamples (bootstrap_seed = 1729), both preregistered and archived. The preregistered power analysis targeted an absolute effect of 0.15 at n = 200 under a baseline assumption $\approx$0.5; because the realized baseline was near-ceiling (0.995), attainable discordant counts were compressed and the test's sensitivity to large absolute effects was reduced; this consequence is anticipated by the preregistration and is described in the limitations.

Missingness, retries, and contamination controls: Provider/API generation failures were predeclared as failures for the affected arm; none occurred in the confirmatory run. The preregistered retry policy allowed a single automatic retry for transient network/API timeouts; all retries and provider-call audit records are archived. Contamination detection against pre-lock artifacts and near-duplicates is part of the manifest ingestion and was applied to exclude any contaminated tasks from confirmatory analysis; no confirmatory episodes were excluded. Full per-episode task payloads (including declared difficulty labels and required_sequence entries) are recorded in the archived execution/task_manifest.jsonl for audit and reanalysis.

Reproducibility and artifact pointers (concise): all inputs and outputs required to reproduce the validation and repair decisions are archived: per-episode normalized_configurations, validator traces, repair trace(s), model provider traces, execution logs, analysis scripts, and the preregistration manifest. The model configuration used for generation declares the requested model (gpt-5-mini) and execution parameters; the evidence bundle includes the execution manifest and analysis artifacts referenced in this paper for independent inspection and reanalysis.

IV. Results

Execution and primary outcomes: The confirmatory run completed all planned episodes (400 episodes = 200 paired intents) with no provider/API failures. Condition-level artifact-backed counts (analysis/condition_summary.csv) show baseline_successes = 199/200 (0.995) and guarded_successes = 200/200 (1.000). The paired contingency (analysis/paired_contingency_table.csv) is n11 = 199, n10 = 1, n01 = 0, n00 = 0; paired difference (guarded - baseline) = 0.005.

Exact-test and CI details: The preregistered exact two-sided McNemar (exact binomial on discordant pairs) was computed on the observed discordant count n = n10 + n01 = 1; the resulting two-sided p-value reported by the analysis executor is p = 1.0 (analysis/results.json). Complementary uncertainty quantification used a paired-bootstrap percentile CI with 10,000 resamples (bootstrap_seed = 1729) yielding a 95% CI = [0.0, 0.015]. Because the preregistered primary effect-size target (absolute 0.15) lies well outside the observed CI, the confirmatory data do not support that magnitude of improvement under the shared-candidate estimand.

Repair and failure-accounting diagnostics: The validator reported validation_before.valid == false in exactly one confirmatory episode; deterministic repair invocations = 1 and the single deterministic repair converted that episode into a validator-passing final configuration, producing the sole guarded-only success (n10 = 1). Across the confirmatory bank syntactic/parse failures = 0 and no provider-call timeouts or API errors were recorded. Provider-call accounting (deterministic_reconciliation.json and execution/model_configuration.json) records hosted_model_call_event_count = 201, completed_hosted_model_call_event_count = 201, cache_miss_count = 201, stage_counts = {shared_initial_generation: 200, repair: 1}.

Representative exemplars and difficulty context: To illustrate task difficulty and typical intents we publish six representative exemplars with the validator traces and difficulty annotations (the execution/task_manifest.jsonl archive contains the full 200-entry manifest). The six exemplars included in the evidence bundle show difficulty labels: 1 medium (shutdown_configure_restore pattern), 2 hard (make-before-break uplink switchover; paired atomic MTU change), and 3 very_hard (safe-access/service-transfer and rolling migration patterns). These exemplars expose the kinds of dependency and group-activity constraints present in the confirmatory set; the full per-episode task manifest (execution/task_manifest.jsonl) contains complete per-task difficulty metadata for re-analysis.

Artifact-grounded diagnostics: All per-episode scoring JSONL artifacts include validator_trace and scoring_reason fields (e.g., execution/scoring/task-00000X-*.json). The paired_contingency_table.csv and condition_summary.csv (analysis/) provide the exact counts used by the primary analysis. The analysis_log.jsonl records bootstrap parameters (resamples = 10,000, seed = 1729) and the exact McNemar computation event. The deterministic_reconciliation.json documents end-to-end episode accounting and verifies that completed_episode_count = 400, complete_pair_count = 200, and that no episodes were excluded for contamination.

Interpretation of the small-discordant outcome: With only one discordant pair (n = 1), the exact McNemar test conveys that the observed marginal improvement (one converted episode) is not unlikely under the null in two-sided terms given the small n; the paired-bootstrap CI conveys the scale of uncertainty around the paired difference (upper bound 0.015). Practically, these diagnostics show that deterministic repair-on-the-same-candidate delivered an auditable, low-cost correction in exactly one case in this curated confirmatory bank; in higher-error regimes or under independent-resampling estimands the repair yield and statistical sensitivity could differ substantially.

Sensitivity notes and archived artifacts for reanalysis: The preregistered missingness plan treated provider/API failures as failures for the arm; because none occurred the primary analysis is not affected by missingness imputation. The preregistered power plan targeted a 0.15 absolute improvement assuming baseline≈0.5; the realized near-ceiling baseline compresses discordant counts and reduces power to detect large absolute effects under this estimand. All relevant artifacts for re-analysis are archived (execution/*, analysis/*, and the full task manifest), enabling investigators to recompute alternative estimands (e.g., independent-resampling) or to inspect the single repaired episode and validator traces for mechanistic insight.

V. Discussion

We expand the operational interpretation and artifact-grounded diagnostics to help practitioners evaluate when a guarded, same-candidate GVR gate is worth deploying.

1) Failure-mode diagnostics and repair yield. The archived validator traces show that across the 200 confirmatory intents the validator emitted zero SYNTAX_PARSE_ERROR events and only one episode with violation_count>0 that the deterministic repair operator classified as repairable under the preregistered mapping. That single episode illustrates two points: (a) the dominant failure modes in this curated bank were not low-level parsing errors but rather missing or misordered commands that the validator could deterministically resolve when the task payload supplied an explicit required_sequence; (b) the deterministic repair yield is therefore highly dependent on task design—when intents include explicit required_sequence elements the operator can deterministically insert or reorder commands with high confidence. The repair trace for the single invoked repair records the deterministic transformation (insertion of a missing shutdown/restore pair at canonical positions) and the validator re-run that produced validation_after.valid = true; both the trace and provider-call audit are archived for inspection.

2) Cost–yield trade-offs under hosted-call budgets. Execution accounting (model_calls_used = 201; shared_initial_generation = 200; repair-stage calls = 1) quantifies the low hosted-call cost of the shared-candidate estimand: a single extra deterministic repair invocation (no additional LLM calls in our run) sufficed to reach 100% guarded success in this bank. In contexts where provider cost or throttling is binding, this design gives nearly full assurance at minimal extra call cost. By contrast, an independent-resampling estimand (two or more independent LLM generations per intent) would increase hosted calls proportionally; the archived per-episode shared_initial response SHA values facilitate counterfactual replays to estimate expected incremental yield from resampling without incurring provider costs by simulating alternative-generation distributions from stored LLM completions.

3) Interpreting the near-ceiling baseline and statistical sensitivity. The near-ceiling baseline (199/200) produced only one discordant pair (n = n10+n01 = 1), which forces the exact-binomial McNemar test to reflect extreme small-n behavior (two-sided p = 1.0 under the observed ordering). The exact McNemar formulation used (binomial test on discordant pairs) is appropriate for such small n because the common large-sample chi-squared approximation is invalid. The paired-bootstrap CI (10,000 resamples; seed = 1729) provides a complementary finite-sample uncertainty interval [0.0,0.015] for the absolute paired difference. Practitioners should therefore interpret our null inferential finding not as evidence that repairs never help, but as evidence that for this curated task bank and single-model generation the marginal improvement achievable by a conservative deterministic repair-on-the-same-candidate is small and auditable. Where baseline error rates are higher, the same guarded machinery would produce larger discordant counts and greater statistical power to detect meaningful improvements.

4) Validator design implications. The three-stage deterministic validator demonstrates operational value beyond pass/fail classification: it emits structured violation codes (MISSING_REQUIRED_COMMAND, DEPENDENCY_VIOLATION, PRO-

TECTED_INTERFACE_MODIFICATION) and quantitative difficulty metadata (parsed_command_count, observed_touched_field_count, required_command_count). These structured outputs enabled our repair operator to apply targeted deterministic actions only when the mapping from violation code to transformation was unambiguous. For production pipelines, we recommend exposing the validator's violation taxonomy to policy layers (e.g., escalation thresholds for HUMAN_REVIEW when violations fall outside the repairable set) and recording violation-code histograms over time for operational monitoring; the archived analysis/contamination_summary.csv and per-episode validator_traces provide the raw counts needed to implement such monitoring.

5) Emulation fidelity and external generalizability. The deterministic emulator applied the normalized_configuration to a Dockerized Mininet+FRR environment and compared observed final_state to expected_final_state. While this provides deterministic, reproducible validation for the control-plane state modeled, the emulator does not model device-specific firmware behaviors, asynchronous control-plane convergence timing (e.g., BGP timers), or vendor command dialect subtleties. The single repair that succeeded did so by addressing a syntactic/ordering gap that is emulator-visible; failures attributable to runtime dynamics would not necessarily be repairable by deterministic static transforms. Users should treat emulator success as necessary but not sufficient evidence for safe production deployment without device-in-the-loop testing.

6) When a guarded same-candidate gate is most appropriate. Based on artifact-grounded evidence, we recommend the shared-candidate GVR gate when: (a) hosted-call budgets are constrained; (b) intents are well-structured with explicit required_sequence or workflow policies; and (c) the validator's repairable-family mapping covers the common failure modes (insertions/reorderings/preservation). When intents are open-ended, diverse in CLI dialect, or when high baseline error rates are observed in validator logs, independent resampling or LLM-invoked repair loops—despite higher provider cost—may be necessary. The execution manifest and provider_call_audit in the archived bundle offer concrete accounting numbers for such cost–benefit calculations.

7) Artifact-grounded reproducibility and next-step diagnostics. We provide all artifacts required to reproduce the run and to run targeted diagnostic re-analyses: per-episode shared_initial responses (execution/responses/*), validator traces (execution/scoring/*.json), repair trace (single invoked repair), and the full task manifest (execution/task_manifest.jsonl; SHA-256: eb2f045f 8a6214eb6d814e16f5d95e0120bd89973951b6fe14084bda 4e3efb4a). Researchers can re-run paired analyses with alternate estimands (independent-resampling) or simulate resampling yield by sampling from archived candidate pools. The archived analysis/paired_contingency_table.csv and analysis/paired_difference_figure.svg provide the exact contingency counts and paired-difference diagnostics used in this paper.

Taken together, these diagnostics show that deterministic validators paired with conservative, auditable deterministic repairs can provide a low-cost assurance gate that is effective when baseline generator quality is high and intents are structured. The approach is not a universal substitute for higher-cost remediation strategies (resampling, LLM-guided repair, or human-in-the-loop review) in settings with higher baseline error rates, multivendor CLI diversity, or dynamic runtime failure modes.

VI. Limitations

- Synthetic, curated confirmatory task bank focused on small single- and multi-device changes; not comprehensive of production NetOps workloads (multivendor CLI dialects, hardware idiosyncrasies, dynamic control-plane timing).
- Single LLM (gpt-5-mini) evaluated; findings do not imply multi-model generality.
- Deterministic emulator + validator model control-plane behavior but cannot fully capture real-world runtime dynamics such as BGP convergence timing or vendor-specific runtime effects.
- Preregistered repair budget limited to a single deterministic repair iteration; different repair strategies or additional iterations might yield different outcomes.
- Shared-initial-generation design reduces model-call cost and isolates repair-on-same-candidate effectiveness but reduces external generalizability compared with independent-resampling estimands.
- Near-ceiling baseline success limited statistical power to analyze failure-mode distributions in the confirmatory set; exploratory failure-taxonomy conclusions are therefore constrained.
- Adversarial or out-of-distribution intents (e.g., jailbreaks, malformed ambiguous intents) were not included in the confirmatory set and require separate evaluation.
- Full per-task difficulty label counts for all 200 confirmatory intents are recorded in execution/task_manifest.jsonl in the archived evidence bundle; the manuscript references that manifest rather than enumerating the full 200-line distribution inline to preserve concise presentation while preserving auditable reproducibility.

VII. Conclusion

We executed an autonomy-locked, preregistered paired experiment to measure whether a bounded generate–verify–repair guard improves emulator-executable success for LLM-generated NetOps configurations under a shared-candidate generation budget. Ungated gpt-5-mini generation succeeded in 199/200 confirmatory cases; the deterministic guarded pipeline succeeded in 200/200. The observed paired difference = 0.005 (exact McNemar two-sided p = 1.0; paired-bootstrap 95% CI [0.0, 0.015]) does not support the preregistered 0.15 absolute-effect target because the baseline was near-ceiling.

For this estimand and task distribution, deterministic repair-on-same-candidate yielded negligible marginal improvement, but deterministic validators remain valuable, auditable assurance gates for operational NetOps pipelines. Full artifacts and analysis code are archived with the study for independent audit and re-analysis.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock defined by the initial master prompt archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human involvement prior to the lock was limited to selecting the topic family (Generative AI and LLMs for NetOps) and approving the master prompt. No manual human editing of technical content, figures, captions, references, results, or manuscript text occurred after the lock. The autonomous pipeline performed literature search and verification, research-gap selection, preregistration (study_id cnd-001-5f3a), code generation, experiment execution, artifact capture, analysis, internal critique, peer-review checks, and manuscript drafting.

Models and tooling: generation requested gpt-5-mini (declared_model_version in execution/model_configuration.json). Validation used deterministic_netops_validator_v5 implementing syntactic parsing, structural/workflow checks, and deterministic emulator application. Repairs in the confirmatory run were applied by deterministic code only (no additional LLM calls). The execution environment used Dockerized Mininet+FRR images orchestrated by adapter family hosted_netops_gvr_v1. Preregistered and enforced settings (not altered post-lock) include: primary estimand = paired_success_rate_difference_guarded_minus_baseline; confirmatory sample = n=200 paired intents; maximum model-call budget = 400; repair budget <=1 deterministic repair iteration; primary analysis = exact McNemar two-sided (exact-binomial on discordant pairs); bootstrap CIs = 10,000 resamples, seed = 1729. Execution accounting (artifact-backed): the run completed 400 episodes with model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). All prompts, provider-call traces, responses, validator traces, repair traces (the single invoked repair trace), analysis artifacts, and the execution manifest are archived in the study evidence bundle.

REFERENCES

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318

[3] Sanjay Mishra, Divya Chukkapalli, Ganesh R. Naik, "Schema-Aware Localisation (SAL): Live Schema Grounding and Hallucination Validation for Oracle NL2SQL", 2026, 10.2139/ssrn.6909831

[4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729

[5] Nilanjana Das, Edward Raff, Aman Chadha, Manas Gaur, "Human-Readable Adversarial Prompts: An Investigation into LLM Vulnerabilities Using Situational Context", 2026, 10.1145/3815174